

Fused Triton Kernels for 4-Bit and Ternary Weight-Only Inference: A TinyLlama-1.1B Case Study

Anshuman Agrawal

University of Petroleum and Energy Studies, Dehradun, India

Abstract

Autoregressive decoding repeatedly reads model weights from device memory, which makes weight compression useful on accelerators with limited memory capacity and bandwidth. This paper studies two custom Triton implementations for TinyLlama-1.1B: a zero-centered 4-bit format and a ternary format stored with two bits per weight. Both kernels load packed INT32 weights, decode them on chip, reconstruct FP16 tiles, and call `tl.dot()` with FP32 accumulation. On an NVIDIA T4, the 4-bit implementation reduced allocated inference memory from 2.21 to 1.05 GB and obtained a sampled WikiText-2 perplexity of 18.90, compared with 17.77 in FP16. In the tested software stack, it generated 21.61 tokens/s, versus 20.56 tokens/s for bitsandbytes NF4 v0.49.2. The ternary representation used 2.125 effective bits per weight, including FP16 group scales, and reduced memory to 0.80 GB. Its quality, however, collapsed under naive groupwise absmean ternarization: sampled perplexity rose to 66,390 and generation became repetitive. A post-quantization affine adaptation that trained 788,480 parameters, about 0.07% of the nominal model size, for 500 steps reduced perplexity to 4,766 but did not restore useful text. These results are a case study of a simple ternary post-training baseline rather than a limit on ternary quantization in general. They provide an implementation-level reference for comparing more advanced activation-aware and optimization-based methods.

Keywords: large language models, weight quantization, Triton, GPU kernels, ternary quantization, post-training quantization

1 Introduction

For batch-one autoregressive decoding, each generated token passes through the model’s linear layers while reusing only a small activation state. The weights must still be read for every step. On memory-constrained accelerators, this makes weight storage and memory traffic central deployment costs. An FP16 model requires roughly two bytes per parameter before accounting for the key-value cache and runtime buffers, so even comparatively small language models can leave little room for longer contexts or larger batches.

Post-training quantization (PTQ) addresses this cost without changing the original training procedure. Four-bit methods are now well established: GPTQ uses second-order information to reduce rounding error, AWQ protects activation-salient weights, and NF4 uses a non-uniform codebook designed for approximately normal weight distributions [1–3]. More elaborate schemes such as AQLM and QuIP# also show that useful models can be compressed below three bits per weight when the quantizer is designed around vector structure, incoherence, or learned codebooks [4, 5].

Ternary models restrict weights to $\{-1, 0, +1\}$. BitNet and BitNet b1.58 demonstrated that this constraint can work when it is part of training [6–8]. The question studied here is narrower. We ask what happens when a conventional FP16 checkpoint is converted with a direct groupwise quantizer and executed through a custom Triton path, without the activation-aware optimization, layer reconstruction, or extended retraining used by stronger ternary methods.

This paper makes four contributions:

- We implement weight-only W4A16 and 2-bit-packed ternary W2A16 linear layers in Triton, with on-chip unpacking and dequantization.

- We report end-to-end allocated memory, decode throughput, sampled WikiText-2 perplexity, and standalone kernel latency for TinyLlama-1.1B on an NVIDIA T4.
- We document a clear negative result for naive groupwise absmean ternarization and test whether a small post-quantization affine adaptation can recover quality.
- We separate the measured result from broader claims about ternary PTQ and relate the failure to newer methods that use asymmetric grids, activation-aware fitting, reordering, and learned modulation.

2 Related Work

2.1 Four-Bit and Extreme PTQ

GPTQ quantizes weights by using approximate second-order information to control the output error introduced by rounding [1]. AWQ instead uses activation statistics to identify and protect important weight channels [2]. QLoRA introduced NF4 and double quantization for memory-efficient fine-tuning, and NF4 is also widely used as a weight-only inference format in bitsandbytes [3]. At lower bit widths, AQLM represents weights through additive codebooks, while QuIP# combines incoherence processing with lattice codebooks and fine-tuning [4, 5]. These methods are relevant because they show that storage width alone does not determine model quality; the quantization objective and representation matter just as much.

2.2 Ternary Training and Ternary PTQ

BitNet and BitNet b1.58 train transformer weights under binary or ternary constraints rather than applying those constraints after training [6, 7]. BitNet b1.58 2B4T later demonstrated a native ternary model at the two-billion-parameter scale [8]. TernaryLLM uses learnable ternarization and feature distillation to recover information lost at W1.58A16 precision [9].

Recent work has also moved ternarization into the post-training setting. PT²-LLM uses asymmetric ternary fitting, activation-aware grid alignment, and structural reordering [10]. TWLA combines ternary weights with low-bit activations through asymmetric fitting, rotations, and mixed-precision activation treatment [11]. CAT-Q uses learnable modulation and softened ternarization on a small calibration set [12]. These results mean that a failed symmetric absmean baseline should not be read as evidence that ternary PTQ is generally infeasible. Our contribution is an implementation and failure analysis of that simple baseline on a small model.

2.3 Low-Precision GPU Kernels

Triton provides a Python-level language for tiled GPU kernels and allows non-standard packing schemes to be expressed without writing architecture-specific CUDA assembly [13]. Our kernels use FP16 activations and reconstruct FP16 weight tiles before `tl.dot()`. This differs from Microsoft’s public BitNet GPU kernel, which targets W2A8 inference and uses packed integer decoding together with the `dp4a` instruction [14]. The distinction matters: ternary storage does not by itself imply ternary arithmetic on current GPUs.

3 Methodology

3.1 Problem Setting

Let $X \in \mathbb{R}^{M \times K}$ be an FP16 activation matrix and $W \in \mathbb{R}^{N \times K}$ the weight matrix of a linear layer. The output is

$$Y = XW^T. \quad (1)$$

We quantize W in groups of $G = 128$ consecutive values along the input dimension. Scales are stored in FP16. Bias terms, when present, remain in FP16.

3.2 Zero-Centered 4-Bit Quantization

For a group g with weights w_i , we compute

$$s_g = \max_{i \in g} |w_i|. \quad (2)$$

For nonzero s_g , each weight is encoded as

$$u_i = \text{clip} \left(\text{round} \left(7.5 \frac{w_i}{s_g} + 8 \right), 0, 15 \right), \quad (3)$$

where u_i is an unsigned 4-bit code and code 8 represents zero. Dequantization reconstructs

$$\hat{w}_i = \frac{u_i - 8}{7.5} s_g. \quad (4)$$

Groups with $s_g = 0$ are encoded as zeros. Eight codes are packed into one INT32 word. With one 16-bit scale per 128 values, the effective storage cost is

$$4 + \frac{16}{128} = 4.125 \text{ bits per weight.} \quad (5)$$

3.3 Ternary Quantization and Packing

For the ternary path, the group scale is the mean absolute value,

$$\gamma_g = \frac{1}{G} \sum_{i \in g} |w_i|. \quad (6)$$

For nonzero γ_g , the ternary code is

$$t_i = \text{clip} \left(\text{round} \left(\frac{w_i}{\gamma_g} \right), -1, 1 \right), \quad (7)$$

with reconstruction $\hat{w}_i = \gamma_g t_i$. The values 0, +1, -1 are stored as 00, 01, and 10; 11 is unused. Sixteen codes are packed into one INT32 word. The resulting fixed-width representation costs

$$2 + \frac{16}{128} = 2.125 \text{ bits per weight.} \quad (8)$$

The value $\log_2 3 \approx 1.585$ bits is a code-space lower bound for a three-state alphabet. It is not the storage cost of this implementation, which uses two fixed bits per ternary value before scale overhead.

3.4 Fused Triton Linear Layer

Each kernel processes the output in two-dimensional tiles. For every step along K , it loads packed INT32 words and FP16 scales from global memory, decodes the selected 4-bit or 2-bit fields with shifts and masks, reconstructs an FP16 weight tile in on-chip memory, and calls `tl.dot()` against the FP16 activation tile. Partial products accumulate in FP32 and are cast to FP16 at the output. The 4-bit path decodes eight values per packed word; the ternary path decodes sixteen. Five candidate tile configurations are supplied to `@triton.autotune`, which selects a configuration for each matrix shape.

This W2A16 design reduces stored weight bytes but still performs a floating-point dot product after reconstruction. Tensor Core use depends on the matrix shape and the compiler’s lowering of `tl.dot()`; the implementation does not assume a native three-state matrix instruction.

3.5 Model Integration

`Linear4Bit` and `LinearTernary` are drop-in `nn.Module` replacements for eligible `nn.Linear` layers. Packed weights and scales are registered as non-trainable buffers. A model-tree traversal replaces linear layers while leaving embeddings, the language-model head, and excluded projection layers in FP16. The original FP16 weight is released after replacement to reduce conversion-time memory pressure.

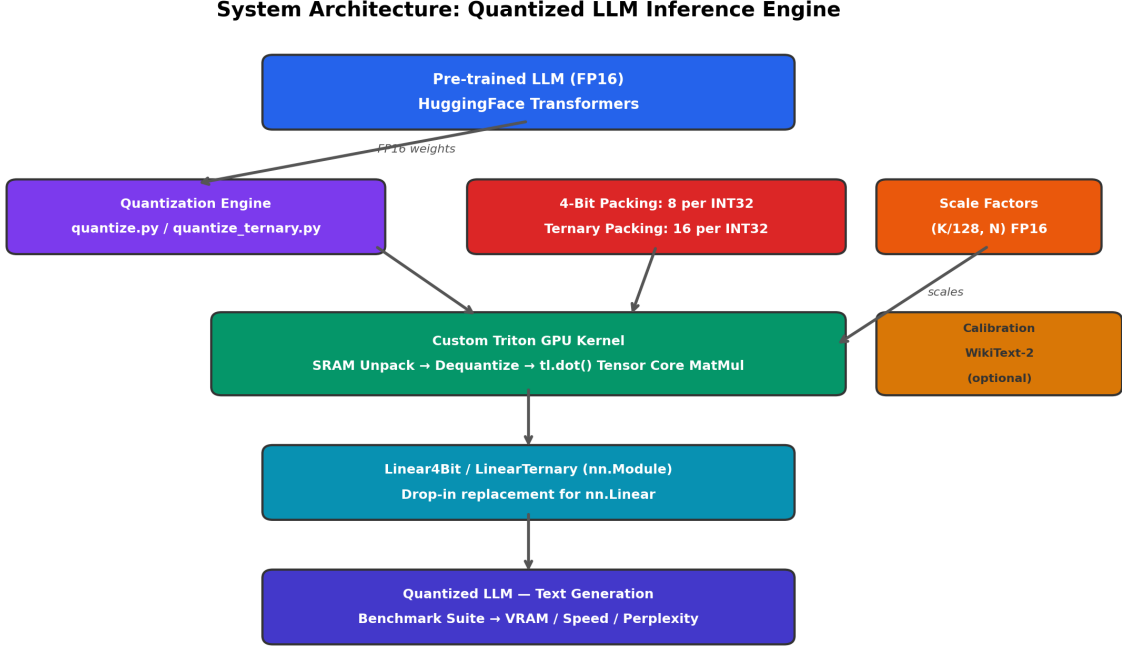


Figure 1: System architecture used in the experiments. FP16 checkpoint weights are quantized on the host, packed into INT32 buffers, transferred with FP16 group scales, and consumed by custom Triton linear layers.

3.6 Post-Quantization Affine Adaptation

To test whether a small amount of optimization can recover ternary quality, each ternary linear output y is followed by a learned per-output scale a and shift b :

$$y' = a \odot y + b. \quad (9)$$

The packed ternary weights remain frozen. Only a and b are trained with next-token cross-entropy on WikiText-2 training text. This adds 788,480 trainable FP32 parameters, approximately 0.07% of a nominal 1.1-billion-parameter model. We use a learning rate of 10^{-5} and run 500 optimization steps.

4 Experimental Setup

4.1 Hardware, Software, and Model

All measurements were made on one NVIDIA T4 exposed through Google Colab. The board has 16 GB of GDDR6 memory and 300 GB/s nominal memory bandwidth; the runtime exposed approximately 15 GB to the process [18]. The reported software stack was Python 3.10, PyTorch 2.10, Triton 3.2, CUDA 12.8, Hugging Face Transformers 5.5.4, and bitsandbytes 0.49.2.

The target checkpoint was `TinyLlama/TinyLlama-1.1B-Chat-v1.0`, a 1.1-billion-parameter model based on the Llama architecture [16]. We compared four configurations: FP16, Triton 4-bit, Triton ternary, and bitsandbytes NF4 v0.49.2.

The bitsandbytes result is tied to that exact version and setup. Version 0.50.0, released after these measurements, introduced new fused 4-bit dequantization-and-GEMM kernels, including kernels for Turing GPUs [15]. The numbers in this paper should therefore not be treated as a benchmark of current bitsandbytes releases.

Table 1: Model-level measurements on TinyLlama-1.1B and an NVIDIA T4. Throughput is mean $\pm$ standard deviation over three subprocess trials. Perplexity is a 50-example WikiText-2 estimate; lower is better.

Configuration	Memory (GB)	Tokens/s	Sampled PPL
FP16	2.21	32.17 ± 0.40	17.77
Triton 4-bit	1.05	21.61 ± 0.38	18.90
Triton ternary	0.80	21.55 ± 0.34	66,390.29
bitsandbytes NF4 v0.49.2	0.88	20.56 ± 0.23	18.24

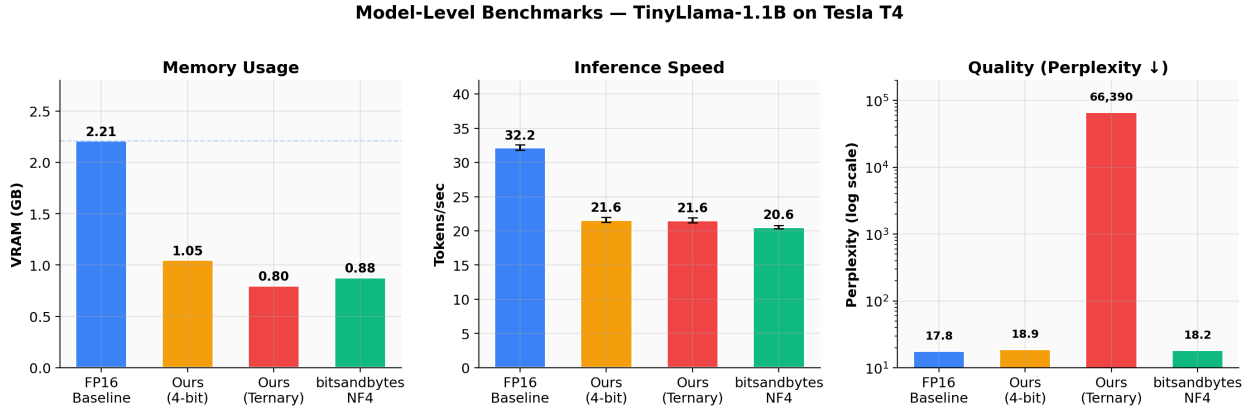


Figure 2: Model-level allocated memory, decode throughput, and sampled WikiText-2 perplexity. The perplexity panel uses a logarithmic scale.

4.2 Metrics

Allocated memory. We reset CUDA peak-memory statistics after quantization, run a forward pass, and report `torch.cuda.max_memory_allocated()`. This measures allocations tracked by PyTorch during inference; it is not a board-level measurement of every byte reserved by the CUDA runtime.

Decode throughput. Throughput is measured during greedy generation of 128 new tokens. Two warm-up runs precede five timed runs. Each configuration is evaluated in three independent subprocess trials, and the table reports the mean and standard deviation across those trials. CUDA synchronization is performed around timed regions.

Language-model quality. Perplexity is estimated as the exponential of mean cross-entropy over 50 examples sampled from the WikiText-2 test split [17]. Because this is a sampled evaluation rather than the standard full-corpus protocol, the absolute values should not be compared directly with full WikiText-2 results reported elsewhere. Four fixed prompts covering code generation, factual explanation, optimization, and quantization were also inspected qualitatively.

5 Results

5.1 Model-Level Comparison

5.2 Four-Bit Result

The 4-bit implementation reduced allocated memory from 2.21 to 1.05 GB, a 52.5% reduction. Sampled perplexity increased by 1.13 points, from 17.77 to 18.90. Relative to bitsandbytes NF4 v0.49.2, the custom path was 5.1% faster in the measured decode test, but it used 0.17 GB more allocated memory and had

Table 2: Storage cost for the implemented weight formats. Scale overhead assumes one FP16 scale per group of 128 weights. Model-level memory also includes FP16 layers and runtime buffers.

Format	Code bits	Scale bits	Effective bits	Ideal ratio
FP16	16.000	0.000	16.000	1.00×
4-bit packed	4.000	0.125	4.125	3.88×
Ternary, 2-bit packed	2.000	0.125	2.125	7.53×

Table 3: Standalone Triton 4-bit latency compared with `torch.matmul` backed by cuBLAS. The ratio is cuBLAS latency divided by custom-kernel latency; values below one mean the custom kernel is slower.

M	K	N	cuBLAS (ms)	Triton (ms)	Ratio
1	2048	2048	0.054	0.121	0.45×
1	2048	8192	0.177	0.323	0.55×
1	8192	2048	0.227	0.332	0.68×
32	2048	2048	0.054	0.173	0.31×
32	2048	8192	0.134	0.570	0.23×
128	2048	2048	0.062	0.570	0.11×

0.66 higher perplexity. It was also 32.8% slower than FP16. No configuration therefore dominates all three reported metrics.

The 5.1% difference should not be attributed to kernel fusion alone. The comparison changes the quantization grid, storage format, module implementation, and kernel-dispatch path at the same time. We did not collect profiler traces or implement an unfused baseline using the same zero-centered 4-bit quantizer, so the experiment does not isolate the cause of the throughput difference.

Across the four qualitative prompts, the 4-bit model produced coherent responses with no obvious failure of the kind seen in the ternary model. This observation is informal and is not a substitute for downstream task evaluation.

5.3 Ternary Result

The ternary implementation reduced allocated memory to 0.80 GB, a 63.8% reduction relative to FP16. Its throughput, 21.55 tokens/s, was effectively the same as the 4-bit path within the observed trial variation. The quality result was very different: sampled perplexity increased to 66,390.29, and generation collapsed into repetitive token fragments and punctuation patterns.

Repeated evaluations returned the same perplexity because the quantizer and evaluation path were deterministic. That repeatability does not test robustness to alternative calibration samples, thresholds, group sizes, or model checkpoints.

5.4 Storage Cost

The table reports the actual fixed-width representation used by the kernels. Calling this implementation “1.58-bit” would be inaccurate: each ternary value occupies two physical bits, and the FP16 group scale raises the effective cost to 2.125 bits per weight.

5.5 Standalone Kernel Measurements

The custom 4-bit kernel was slower than cuBLAS for every standalone shape tested. This is compatible with the model-level comparison: the cuBLAS baseline multiplies FP16 weights, whereas the model-level bitsandbytes comparison uses a different compressed representation and software path. End-to-end speed cannot be inferred from the standalone matrix multiplication table alone.

Table 4: Held-out sampled perplexity during affine adaptation of the ternary model. The 788,480 correction parameters are approximately 0.07% of the nominal 1.1B model size.

Step	Sampled PPL	Improvement over step 0
0	66,390.29	1.00×
100	20,141.15	3.30×
200	11,182.22	5.94×
300	7,310.62	9.08×
400	5,698.88	11.65×
500	4,766.00	13.93×

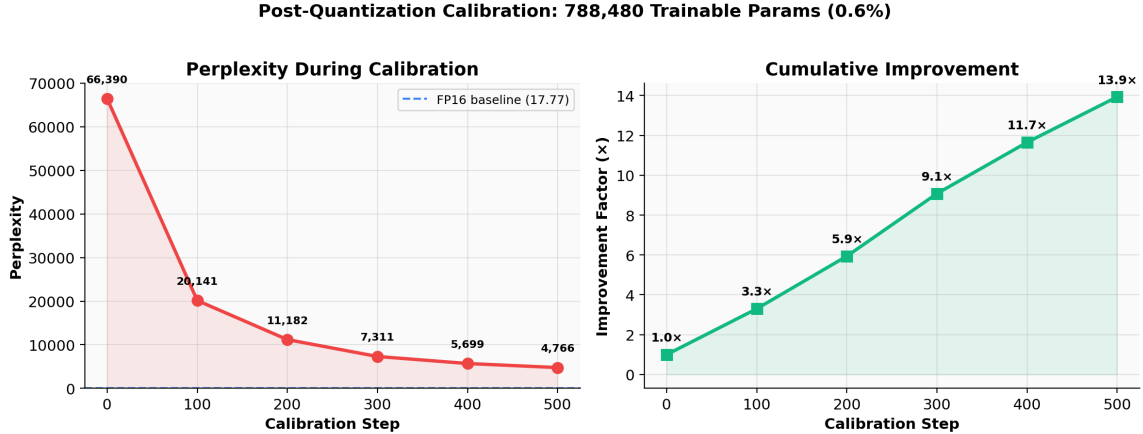


Figure 3: Perplexity and cumulative recovery during post-quantization affine adaptation. Perplexity continued to improve through step 500, but remained far above the FP16 value.

5.6 Post-Quantization Affine Adaptation

The adaptation reduced sampled perplexity by a factor of 13.93, from 66,390.29 to 4,766.00. The adapted model was still about 268 times worse than the FP16 baseline on the same sampled evaluation, and qualitative generation remained incoherent. The result shows that per-output scales and shifts can correct part of the accumulated distribution error, but they are not expressive enough to reconstruct the input-dependent structure removed by this ternary quantizer.

6 Discussion

6.1 What the Ternary Experiment Establishes

The experiment establishes a specific failure mode: groupwise absmean ternarization with a fixed symmetric three-state grid breaks TinyLlama-1.1B under the W2A16 setup used here. It does not establish that all ternary PTQ methods fail. PT²-LLM, TWLA, and CAT-Q use asymmetric grids, activation-aware objectives, reordering, rotations, softened optimization, or learned modulation to address precisely the kinds of asymmetry and outlier sensitivity ignored by a direct absmean projection [10–12].

Our measurements do not identify a single causal layer or statistic. A plausible interpretation, consistent with that later work, is that the simple quantizer introduces structured errors that compound across layers. Testing that interpretation requires layerwise output error, activation-weighted reconstruction error, ternary occupancy, and mixed-precision rescue experiments. Those analyses were not part of the present study.

6.2 What the Affine Adaptation Shows

A learned output scale and shift can change the mean and magnitude of each output channel, but it cannot restore the original input-channel-specific weight pattern. The strong early reduction in perplexity shows that some of the failure is correctable after quantization. The remaining gap shows that this correction family is too limited for deployment-quality recovery on this checkpoint. More expressive alternatives include low-rank adapters, blockwise reconstruction, knowledge distillation, or full quantization-aware fine-tuning.

6.3 Kernel Implications

The ternary format achieves its memory reduction through packing, not through a native ternary matrix instruction. In our W2A16 implementation, values are decoded and reconstructed as FP16 before the dot product. Integer W2A8 kernels such as Microsoft’s public BitNet implementation follow a different route and use integer dot-product instructions [14]. Future performance work should therefore compare complete arithmetic paths, not only nominal weight bit widths.

For the 4-bit path, the end-to-end result against bitsandbytes v0.49.2 is encouraging but narrow. A current comparison should include bitsandbytes 0.50.0 or later, record the selected kernels, separate prefill from decode, and profile memory traffic and launch overhead. It should also include an unfused implementation of the same quantizer so that fusion can be measured independently of format differences.

7 Limitations and Future Work

This study uses one 1.1B chat checkpoint, one T4 GPU, one group size, and one simple thresholding rule. The quality evaluation is based on 50 WikiText-2 examples and four qualitative prompts rather than the full corpus and a standard downstream benchmark suite. Throughput is reported for a single fixed generation setup; prompt length, batch size, prefill latency, and longer-context behavior are not swept. The bitsandbytes baseline is v0.49.2 and does not represent the later fused 4-bit implementation. We also did not collect profiler traces, test multiple random calibration subsets, or evaluate layerwise sensitivity.

The next experiments should vary group size and ternary thresholds, quantize one layer at a time, measure layer-output cosine similarity and mean-squared error, and retain sensitive layers at four or sixteen bits. The benchmark should be repeated on at least one additional 1B–3B checkpoint and on a current consumer GPU. A stronger comparison would implement PT²-LLM, CAT-Q, or another modern ternary PTQ method using the same model and evaluation protocol. On the systems side, prefill and decode should be reported separately, and the W2A16 path should be compared with an integer-activation W2A8 design. Quantization-aware fine-tuning is also a natural extension; Gemma 3 provides a practical example of short QAT schedules for 4-bit checkpoints [19].

8 Conclusion

We implemented fused Triton linear layers for a zero-centered 4-bit format and a 2-bit-packed ternary format, then evaluated them on TinyLlama-1.1B and an NVIDIA T4. The 4-bit path cut allocated inference memory by 52.5% and kept sampled perplexity close to FP16. Under the tested bitsandbytes 0.49.2 setup, it also achieved a modest decode-throughput advantage, although it used more memory than NF4 and was slower than FP16. The ternary path reduced memory by 63.8% but failed under naive groupwise absmean quantization. Training a small affine correction reduced the error substantially without restoring useful generation.

The main conclusion is deliberately narrow: a simple symmetric ternary projection is not a viable drop-in replacement for TinyLlama-1.1B in this setup. The result is best used as a baseline for methods that optimize the ternary grid, account for activations and outliers, or retrain the model under low-precision constraints.

Declaration of Tool Assistance

Generative language tools were used for editorial revision and organization. The authors verified the technical statements, numerical results, references, and conclusions and take full responsibility for the manuscript.

References

- [1] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [2] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” *arXiv preprint arXiv:2306.00978*, 2023.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” *arXiv preprint arXiv:2305.14314*, 2023.
- [4] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, “Extreme compression of large language models via additive quantization,” *arXiv preprint arXiv:2401.06118*, 2024.
- [5] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, “QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks,” *Proc. International Conference on Machine Learning*, 2024.
- [6] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “BitNet: Scaling 1-bit transformers for large language models,” *arXiv preprint arXiv:2310.11453*, 2023.
- [7] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit LLMs: All large language models are in 1.58 bits,” *arXiv preprint arXiv:2402.17764*, 2024.
- [8] S. Ma, H. Wang, S. Huang, X. Zhang, Y. Hu, T. Song, Y. Xia, and F. Wei, “BitNet b1.58 2B4T technical report,” *arXiv preprint arXiv:2504.12285*, 2025.
- [9] T. Chen, Z. Li, W. Xu, Z. Zhu, D. Li, L. Tian, E. Barsoum, P. Wang, and J. Cheng, “TernaryLLM: Ternarized large language model,” *arXiv preprint arXiv:2406.07177*, 2024.
- [10] X. Yan, C. Bao, Z. Li, T. Zhang, K. Yang, H. Qin, R. Xie, X. Sun, and Y. Zhang, “PT²-LLM: Post-training ternarization for large language models,” *Proc. International Conference on Learning Representations*, 2026.
- [11] Z. Zhao, Z. Xu, Z. Chen, X. Hu, Z. Jiang, and D. Yang, “TWLA: Achieving ternary weights and low-bit activations for LLMs via post-training quantization,” *arXiv preprint arXiv:2606.13054*, 2026.
- [12] S. Wang, C. Li, Y. Kang, J. Fan, and A. Yao, “CAT-Q: Cost-efficient and accurate ternary quantization for LLMs,” *arXiv preprint arXiv:2606.26650*, 2026.
- [13] P. Tillet, H. T. Kung, and D. Cox, “Triton: An intermediate language and compiler for tiled neural network computations,” in *Proc. 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19.
- [14] Microsoft, “BitNet inference kernel: W2A8 GPU implementation,” 2026. [Online]. Available: Microsoft BitNet GPU README. [Accessed: 13 Aug. 2026].
- [15] Bitsandbytes Contributors, “bitsandbytes 0.50.0: Faster 4-bit inference, CPU/ROCm/Apple Silicon improvements, and more,” 2026. [Online]. Available: bitsandbytes v0.50.0 release notes. [Accessed: 13 Aug. 2026].
- [16] P. Zhang, G. Zeng, T. Wang, and W. Lu, “TinyLlama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [17] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” in *Proc. International Conference on Learning Representations*, 2017.
- [18] NVIDIA, “NVIDIA T4 Tensor Core GPU datasheet,” 2019. [Online]. Available: NVIDIA T4 datasheet. [Accessed: 13 Aug. 2026].
- [19] Gemma Team, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025.